Response to Reviewers

Manuscript: *Latency-Constrained Request Routing for Interactive LLM Applications: A Multi-Armed Bandit Approach*

Summary of Changes

We thank the Editor and both Reviewers for their constructive and detailed feedback. The manuscript has been substantially revised to address every point raised, and all substantive changes are marked in the revised manuscript for ease of review.

The most important changes are as follows.

1. *Latency-model realism* (R1, Major 1). The simulator now models M/G/1-style queueing delay, continuous batching on the cloud tier, probabilistic cold-start penalties, explicit time-to-first-token plus per-token decoding, and a Pareto-mixed heavy tail for cloud variability. A sensitivity analysis toggling each component is reported in Table 4 and Figure 6. We also added a GPU-calibrated empirical replay on an RTX 5090 in Section 6.9, reported in Figures 8–10.

2. *Complexity-predictor characterisation* (R1, Major 2; R2, Major 3). The predictor is now fully described: architecture (2-layer MLP, 1,187 parameters), training dataset (6,000 synthetic examples across three domains with 5% label noise), training procedure, validation accuracy (80% on a 1,500-example held-out set), per-class confusion matrix, per-domain accuracy, and robustness under feature perturbation. Figure 11 reports the predictor evaluation and Table 6 quantifies the effect of predictor quality on downstream routing.

3. *Theoretical discussion of DATS* (R1, Major 3). Section 4.4 now discusses (a) how DATS preserves the Thompson Sampling regret structure up to a bounded multiplicative constant, (b) the role of the penalty parameter λ and why it does not collapse exploration in practice, (c) convergence of the complexity-conditioned prior, and (d) three potential pathological behaviours with empirical evidence that each is controlled.

4. *Expanded baselines* (R1, Major 4). Three new baselines are added throughout the revision: a FrugalGPT-style cascade, a QoS-driven static heuristic, and a cost-aware scheduler with SLA guardrail. Results for all eleven policies appear in the new Table 2.

5. *Multi-domain evaluation* (R1, Major 5). A new Section 6.6 evaluates the framework on three domains (interactive narrative, customer QA, code generation) using the same predictor and routing policy. The framework generalises well to narrative and customer QA; the code-generation analysis uncovers an inherent limitation of interaction-scale deadlines, which we now discuss as a domain-level finding.

6. *DATS vs. Thompson Sampling addressed directly* (R2, Major 1). The DATS algorithm has been re-designed to incorporate (i) an upper-tail latency estimate $\hat{\ell}_k + \kappa\hat{\sigma}_k$, (ii) a recency-weighted deadline-miss signal, and (iii) an adaptive penalty that scales with sustained misses. A paired bootstrap test over five seeds shows that DATS reduces SLA violations relative to plain Thompson Sampling at p=0.007 (95% CI $[-0.108, -0.022]$ pp). Thompson Sampling is

now included in the non-stationary experiment (Table 3) and the multi-domain experiment (Table 5).

7. *Expanded quality validation* (R2, Major 2). The quality-validation benchmark has grown from 15 to 60 queries across three domains, for 360 graded responses, and the expanded pipeline is released in the repository. We further added a quota-limited Gemini 3.1 re-measurement using `gemini-3.1-flash-lite` as the edge proxy and `gemini-3.1-pro-preview` as the cloud/judge model (Table 8).

8. *Related-work expansion* (R2, Major 4). Section 2 now surveys MAB-based network-routing literature and cites Scarvaglieri et al. (MAGELLAN, GLOBECOM 2024) and Lian et al. (IEEE JSAC, 2025), among others. Table 1 has also been extended to include these works.

9. *Open-source release* (R2, Major 5). The simulator, MLP weights, configuration files, and experiment scripts are released at https://github.com/Sangdon-Park/edgellm-router, with a footnote link at the first mention of the simulator in Section 5.

We respond to each individual comment below. Reviewer comments are reproduced in italics; our responses follow.

Reviewer #1

Major 1 — Latency modelling realism

The latency models used in the simulator are highly simplified (e.g., log-normal network latency and linear inference cost). These abstractions omit several major contributors to real-world end-to-end latency, such as queueing delays, batching pipelines, token streaming behavior, cold-start effects, and cloud service variability. This raises concerns regarding the applicability of the results to practical deployments. The authors should discuss these limitations explicitly and, if possible, evaluate the sensitivity of DATS to deviations from the assumed latency model or validate the model against additional empirical traces.

We have substantially revised the simulator's latency model to include every effect the reviewer mentioned, and have added a sensitivity analysis.

- *Queueing delay* is now modelled as an M/G/1-style queue with per-tier arrival rate, mean service time, and concurrency. The instantaneous utilisation is $\rho = \lambda_a \bar{S}/c$, and per-request waiting time is drawn from an exponential around $\rho\bar{S}/(1-\rho)$ (Section 5). The parameters follow Kleinrock (1975) and Gujarati et al. (OSDI 2020).
- *Continuous batching* on the cloud tier is modelled as a uniform $[0, B]$ wait before service (B=15 ms by default), matching reported behaviour in vLLM and Orca (Section 5).
- *Cold-start* effects are modelled with per-tier idle timers; if the idle gap exceeds the timeout, the next request pays half the cold-start penalty to reflect partial cache warmth (Section 5).
- *Token streaming* is reflected in an explicit time-to-first-token term plus per-token prefill and decode components: $\ell_{\text{infer}} = \ell_{\text{TTFT}} + \alpha_{\text{prefill}} n_{\text{in}} + \alpha_{\text{decode}} n_{\text{out}} + \epsilon$ (Section 5).
- *Cloud service variability* now includes a Pareto heavy-tail mixture: with probability p_{tail}, the sampled network latency is multiplied by an i.i.d. Pareto(1.5) spike (Section 5).

Table 4 (Section 6.5) reports DATS and Thompson Sampling SLA violations and p_{99} latency when each realism component is toggled on and off, plus stress scenarios including `extreme_tail_p10` and `cloud_outage_burst`. DATS remains below or near 1% SLA violations in all settings and ties or beats Thompson Sampling on SLA in every setting, so the headline results are not an artefact of a single latency-model choice. Figure 6 visualises the breakdown.

Section 5 now enumerates explicit limitations: single-tenant queue approximation, static tier capability, streaming treated as a latency effect only, and scalar-quality modelling. To complement the synthetic sensitivity study, Section 6.9 adds a GPU-calibrated empirical replay path on an RTX 5090. The new figures report measured latency, throughput, VRAM, and NVML power for Qwen3 proxy tiers and replay routing policies on empirical traces. Finally, validation of the scalar-quality assumption against LLM-as-judge scores is discussed in Section 6.12 (Table 8), whose benchmark grew from 15 to 60 queries in response to Reviewer 2 Major 2 and now includes an additional Gemini 3.1 re-measurement.

Major 2 — Complexity predictor details

Although the query complexity predictor is central to the routing framework, the paper provides insufficient information regarding its construction and performance. Key missing details include the training dataset size and source, the labeling process and its reliability, classification accuracy, robustness to distribution shift, and the impact of prediction errors on routing performance. Because predictor errors directly affect regret and SLA violations, the authors should

include quantitative evaluation of the predictor and analyze how misclassification influences system performance.

Section 4.1 now provides a complete characterisation of the predictor.

- *Architecture.* 2-layer MLP with a hidden layer of 16 ReLU units, 1,187 trainable parameters, output by softmax cross-entropy.
- *Training data.* 6,000 synthetic examples sampled uniformly from three application domains (narrative, customer QA, code). Each example's six features are drawn from complexity-conditioned distributions: input length, output length, entity density, context utilisation, and turn index all shift systematically with the ground-truth complexity.
- *Labelling reliability.* To emulate the disagreement observed in human-labelling pilots, 5% of training labels are flipped to a neighbouring class during dataset synthesis.
- *Training procedure.* Mini-batch SGD, batch 64, learning rate 0.05, L2 10^{-4}, 60 epochs.
- *Validation accuracy.* 80% three-way accuracy on a 1,500-example held-out set (zero label noise). Per-class recall: 85% (Simple), 70% (Moderate), 84% (Complex).
- *Confusion matrix.* Figure 11 (left) shows that the dominant error mode is Simple↔Moderate, which is benign for routing since both classes map to edge-friendly tiers.
- *Per-domain accuracy.* 77% (narrative), 80% (customer QA), 81% (code); no systematic degradation across domains.
- *Transfer to realistic text queries.* On the 60-query realistic validation bench, the same synthetic-trained predictor reaches 85.0% three-way accuracy; 5-fold cross-validation after adding realistic-query folds gives 83.3%.
- *Robustness.* Figure 11 (right) plots accuracy under Gaussian feature perturbation: 80% at σ=0, 77% at σ=0.05, 73% at σ=0.1, 62% at σ=0.2, 55% at σ=0.3.
- *Impact on routing.* Table 6 (Section 6.10) reports DATS performance with three predictor configurations. Replacing the well-trained predictor (80% accuracy) with a noisy one (55% accuracy) or a uniform prior changes SLA violations by less than 0.15 pp. The predictor therefore provides routing guidance and faster early-phase convergence without being a single point of failure.

The predictor, its training script, and its weights are released in the repository linked from the revised Section 5.

Major 3 — Theoretical justification for reward design

The latency-aware reward formulation in Equation (1) is heuristic in nature, and the paper focuses primarily on empirical validation. There is no theoretical discussion of whether DATS preserves Thompson Sampling regret properties, whether the penalty parameter λ could lead to insufficient exploration, or how complexity-conditioned priors affect convergence. The authors should provide theoretical insight, at minimum discussing limitations, potential pathological behaviors, and stability considerations.

A new Section 4.4, "Algorithmic Properties," now discusses each of the points raised.

(a) Does DATS preserve the Thompson Sampling regret bound? Standard Thompson Sampling regret bounds require two properties: (i) the posterior is updated only from observations of

the chosen arm, and (ii) the arm selected at each round is a function of a posterior sample. DATS preserves (i) verbatim—the Beta posterior is updated by the (possibly deadline-penalised) quality observation. Property (ii) is relaxed: the score s_k combines the posterior sample $\tilde{q}_k$ with deterministic, slowly-varying quantities p_k, m_k, and $\pi(\hat{c}_t, k)$. Because s_k is monotone-increasing in $\tilde{q}_k$ and the reweighting factors are bounded by construction ($p_k \in [0,1]$, $m_k \in [0,1]$, $\pi \in [-0.08, 0.10]$), the arm-selection rule corresponds to a bounded affine reweighting of the posterior samples and inherits the standard regret bound up to a multiplicative constant that depends on the reweighting range. The argument follows the analysis of Russo et al. (2018) extended to a bounded score transformation. The stationary-workload experiments (Section 6.2) already show DATS is empirically close to Thompson Sampling in regimes where the bound is tight; the practical value of DATS lies in the non-stationary and heavy-tail robustness that the bound cannot capture.

(b) Penalty parameter λ and exploration. For small λ, DATS reduces to Thompson Sampling with a mild deadline bias. For large λ, the penalty could in principle collapse exploration onto arms with $p_k \to 1$. In practice $\hat{\sigma}_k$ is non-negligible during warm-up (driven by the prior $\sigma_k^{(0)}$), so p_k is bounded away from 1 for each arm until it has been tried several times, preserving sufficient exploration. Figure 12 confirms that $\lambda \in [0,1]$ smoothly interpolates between quality-first and SLA-first regimes with no visible collapse even at λ=1.

(c) Complexity-conditioned prior and convergence. The prior term $\pi(\hat{c}_t, k)$ adds at most 0.10 to an arm's score. Since the posterior-sample term $\tilde{q}_k$ lies in $[0,1]$, π is non-dominating: once α_k, β_k are informative, $\tilde{q}_k$ dominates π. Its role is therefore to warm-start exploration in a statistically sensible region. Empirically, the prior shortens convergence by roughly 30–50 requests under our traces (Figure 4).

(d) Pathological behaviours. We enumerate three failure modes with mitigations:

- *Stale miss rates:* if traffic is sparse, m_k may remain high after conditions improve. The tail-factor-based p_k partially mitigates this because it recovers as $\hat{\sigma}_k$ shrinks.
- *Predictor drift:* Table 6 shows that replacing the predictor with a noisy or uniform one moves SLA violations by less than 0.15 pp, indicating robust behaviour.
- *Adversarial heavy tail:* an attacker synchronising latency spikes across arms could starve all tiers. This is out of scope, but production deployments would add rate limiting at ingress.

Major 4 — Incomplete baseline comparisons

The experimental comparison is limited mainly to bandit-based baselines (UCB1, Thompson, ε-greedy). To strengthen the practical impact of the results, additional baselines should be considered, including cascade-based routing (e.g., FrugalGPT-style approaches), QoS-driven static heuristics, and cost-aware scheduling policies. Without these comparisons, it is difficult to fully assess the advantage of DATS over existing deployment strategies.

Three new baselines are added and appear in every relevant table and figure.

- *FrugalGPT cascade.* Runs edge first; escalates to cloud if the (simulated) edge confidence drops below a threshold. Implementation: `FrugalGPTCascadePolicy` in `edge_simulator.py`.
- *QoS static.* Rule-based heuristic that uses the running network-latency EMA to prefer edge under degraded networks (`QoSStaticPolicy`).

- *Cost-aware.* Cost-minimising scheduler with an SLA guardrail, in the spirit of SHEPHERD (`CostAwarePolicy`).

Table 2 reports all eleven policies side by side:

Policy	Quality	p_{99}	SLA Viol.	Cost
Cloud-only	0.978	1378 ms	53.4%	\$0.740
FrugalGPT-cascade	0.972	965 ms	18.6%	\$0.325
QoS-static	0.970	500 ms	0.9%	\$0.037
Cost-aware	0.968	720 ms	6.1%	\$0.001
Thompson	0.970	500 ms	0.44%	\$0.097
DATS	0.970	500 ms	0.36%	\$0.097

Two observations are worth highlighting. First, the cascade baseline is competitive on quality but degrades on SLA when the cloud tier's heavy tail dominates, because each cascade escalation pays the full cloud latency. Second, QoS-static is surprisingly strong (0.9% SLA) because the workload is stationary enough that a hand-tuned heuristic using the network EMA keeps up with DATS. However, QoS-static is brittle to regime changes (Section 6.4) and requires per-deployment tuning, whereas DATS adapts automatically.

Major 5 — Limited evaluation generality

All experiments are conducted using a single interactive narrative application workload. This narrow evaluation raises concerns about workload-specific behavior. The authors are encouraged to include at least one additional application domain (e.g., customer service QA, code generation, document processing) and to analyze whether both the complexity predictor and the routing policy generalize across domains.

A new multi-domain evaluation (Section 6.6) extends the framework to three domains: interactive narrative (the original workload), customer QA (shorter, mostly factual), and code generation (longer inputs, heavier compute). The complexity predictor and DATS are trained once on a uniform mixture of all three domains and evaluated per domain.

Table 5 summarises the results:

Domain	Policy	Quality	p_{99}	SLA
narrative	DATS	0.970	500 ms	0.57%
customer-QA	DATS	0.972	586 ms	2.19%
code	DATS	0.967	1631 ms	53.1%

The narrative results replicate the main-table behaviour (SLA 0.57%). On customer QA, DATS and Thompson are within noise of each other (about 2% SLA): the mostly-factual, simple-skewed workload makes edge-first routing already very effective. On code generation, both DATS and Thompson struggle because inputs and outputs are substantially longer—decoding a 300-token code block at 150 tokens/s exceeds 2 s—pushing all routing policies above the 500-ms SLA threshold.

We view the code-generation result as itself a contribution: it clarifies that the adaptive-routing framework assumes interaction-scale requests and highlights code generation as a domain that needs either a larger $L_{\max}$ or streaming partial outputs. We note this explicitly in Section 6.6 and in the Discussion (Section 7).

The complexity predictor generalises well across all three domains (77–81% per-domain accuracy; see Section 4.1).

Major 6 — Presentation and clarity

Several presentation and clarity issues should also be addressed. Some figures suffer from readability problems, such as overlapping error bars. The manuscript should clarify whether posterior updates in DATS rely on full historical data or employ recency weighting. The rationale behind the choice of latency threshold ($L_{\max}$) should be more clearly connected to user experience constraints. Additional details regarding hybrid-tier execution and SLA fallback mechanisms would further strengthen the system description.

We have addressed each of the four sub-points.

Figure readability. Figure 13 (variance analysis) has been redrawn using split SLA axes and per-seed summaries so that the sub-1% policies remain visible next to high-violation baselines. Figure 2 (tier distribution), Figure 7 (multi-domain), and Figure 6 (sensitivity) use offset positioning and zoomed panels so that error bars and labels no longer overlap.

Recency weighting. Section 4.3 now explicitly states that DATS uses two exponentially-weighted moving averages: one for latency mean and standard deviation (coefficient γ=0.55, effective memory $\approx$ 2.2 requests) and one for the deadline-miss indicator (coefficient 0.65). The Beta posterior itself uses the full history of quality observations, as in standard Thompson Sampling. Algorithm 1 has been updated to reflect both EMAs explicitly.

$L_{\max}$ *rationale.* Section 3.1 now grounds $L_{\max}$=500 ms in Nielsen's response-time thresholds: below 100 ms an interaction feels instantaneous, up to 1 s flow is retained but delay is noticed, beyond 1 s flow is disrupted. For conversational LLMs, industry practice places the tolerable ceiling near 500 ms (citing FrugalGPT and RouteLLM), which is the value used throughout the paper.

Hybrid tier and SLA fallback. Section 4.3 now explains that the hybrid tier runs an edge-drafting pass and, if the remaining budget $L_{\max} - \ell_{\text{edge-draft}}$ exceeds 180 ms, starts a cloud-refinement pass in parallel. The refinement is capped by the remaining budget so that the hybrid response is always delivered within $L_{\max}$ or returns the edge draft. In the simulator this is modelled explicitly; in deployment it is implemented with a best-effort async future that is cancelled when the timer expires.

Reviewer #2

Major 1 — DATS does not clearly outperform Thompson Sampling

The most significant concern is that the proposed DATS algorithm does not clearly outperform plain Thompson Sampling on the primary metrics discussed in Section 6.1. Looking at Table 2, Thompson Sampling achieves lower SLA violations (1.2% vs 1.7%) and lower p_{99} latency (521 ms vs 560 ms) than DATS. The authors should address this directly. Compounding this, Thompson Sampling is entirely absent from the non-stationary experiment in Figure 5 and Table 3, precisely the scenario where DATS should demonstrate its advantages. The authors must provide an explanation to this discrepancy and update both the figure and the table.

This is a fair concern, and we agree that the original manuscript did not make a convincing case. We have addressed it in three ways: (i) re-designed DATS so that it has genuine algorithmic differences from Thompson Sampling, (ii) included Thompson Sampling in every subsequent experiment, and (iii) added a statistical test.

(i) DATS re-design. The revised DATS algorithm (Algorithm 1, Section 4.3) differs from plain Thompson Sampling in three concrete ways:

- *Tail-aware deadline probability.* The deadline probability uses an upper-tail latency estimate $\hat{\ell}_k + \kappa\hat{\sigma}_k$ (with κ=1.5) rather than the mean alone, so that heavy-tail spikes reduce the arm score.
- *Recency-weighted miss signal.* A second EMA tracks the per-tier deadline-miss rate m_k with coefficient 0.65, and the arm score is penalised by $w_m m_k$ (w_m=1.6).
- *Adaptive penalty.* The deadline penalty is scaled by $1+2\max_k m_k$ so that sustained misses accelerate the switch to safer tiers.

(ii) Results on the revised Table 2 (5 seeds, 10,000 requests each).

Policy	Quality	p_{99}	SLA Viol.
Thompson	0.9696 ± 0.0002	500 ms	0.44% ± 0.06%
DATS	0.9696 ± 0.0002	500 ms	0.36% ± 0.10%

DATS now achieves a lower SLA-violation rate than Thompson Sampling at equivalent quality.

(iii) Statistical test. A paired bootstrap test with 20,000 resamples over the 5 seeds returns:

$$\begin{aligned} \text{mean SLA difference (DATS} - \text{TS)} &= -0.076 \text{ pp}, \\ 95\% \text{ bootstrap CI} &= [-0.108,\ -0.022] \text{ pp}, \\ \text{two-sided } p\text{-value} &= 0.007, \\ \text{quality difference} &= -7 \times 10^{-5}. \end{aligned}$$

The CI excludes zero and $p < 0.01$; DATS is therefore significantly better than plain Thompson Sampling on SLA violations while indistinguishable on quality. Section 6.7 ("Statistical Significance") reports this test.

Non-stationary experiment now includes Thompson Sampling. Table 3 has been updated:

Policy	Before (stable)	After (unstable)
Complexity-static	0.972 / 13.4%	0.972 / 22.9%
UCB1	0.969 / 4.2%	0.969 / 3.4%
Thompson	0.970 / 0.68%	0.970 / 0.56%
DATS	0.970 / 0.52%	0.970 / 0.38%

Figure 5 has been regenerated to include the Thompson Sampling bars. DATS still achieves the lowest SLA-violation rate in both phases, with the advantage coming from the tail-aware and miss-rate components that detect the regime shift a few requests earlier than Thompson's Beta posterior alone.

The Main Results text (Section 6.2) and bulleted summary have been rewritten to be more honest and precise: DATS and Thompson are very close on stationary workloads (both excellent); DATS's advantage is statistically significant and becomes clearer under non-stationary conditions, heavy-tail stress, and domain shifts.

Major 2 — 15-query test set too small

Moreover, the 15-query test set used in Section 6.8 is insufficient to establish that results transfer to real deployments.

The benchmark has been expanded from 15 to 60 queries, covering three application domains (20 queries per domain) across three complexity levels, with three repetitions per (query, tier) pair, for 360 graded responses in total. Section 6.12 ("Simulator Assumption Validation") reports the updated numbers:

Tier	Complexity	Measured	Assumed	Diff
Edge	Simple	0.978 ± 0.012	0.980	-0.002
Edge	Moderate	0.968 ± 0.020	0.970	-0.002
Edge	Complex	0.932 ± 0.050	0.950	-0.018
Cloud	Simple	0.989 ± 0.010	0.990	-0.001
Cloud	Moderate	0.978 ± 0.012	0.980	-0.002
Cloud	Complex	0.951 ± 0.075	0.970	-0.019

All measurements fall within ± 0.02 of the assumed values, and confidence intervals are tightened by roughly a factor of two compared with the original 15-query set. The expanded pipeline is released in `experiments_quality_validation.py` and supports both an API-backed path (requires Gemini API keys) and a reproducible synthetic path. We additionally re-measured a balanced 36-response subsample with Gemini 3.1 models, using `gemini-3.1-flash-lite` for the edge proxy and `gemini-3.1-pro-preview` as the cloud/judge model. This fresh check preserves the same conclusion: all Gemini 3.1 measurements are within ± 0.03 of the assumed simulator quality values.

Clarification on the role of Section 6.8 vs Section 6.12. Section 6.12 (Simulator Assumption Validation, n=60 queries $\times$6 conditions $\times$3 repetitions = 360 graded responses, plus the Gemini 3.1 subsample) is the section that addresses this concern: it provides a statistically grounded validation of the simulator's quality parameters. Section 6.8 (End-to-End Deployment Validation, n=9 routing decisions per policy) serves a separate, complementary purpose: it is a qualitative sanity check confirming that DATS's routing behaviour transfers from the simulator to live API endpoints. The small n in Section 6.8 precludes statistical conclusions, which is why the text explicitly states that the simulator experiments remain the primary controlled evidence. In short, Section 6.12 validates *what* the simulator assumes (quality values); Section 6.8 validates *that* the routing algorithm works against real APIs.

Major 3 — Complexity predictor training data not described

Regarding the complexity predictor described as a core contribution in Section 4.1, its training data is never described anywhere in the paper — the authors state only that it is "trained on labeled examples" without specifying how many, where they come from, how labeling was

performed, or what classification accuracy the predictor achieves. The authors should provide a proper characterisation and evaluation of the complexity predictor.

Please see our response to Reviewer 1 Major 2. In brief, Section 4.1 now provides the architecture (2-layer MLP, 1,187 parameters), training data (6,000 synthetic examples across three domains with 5% label noise), training procedure (mini-batch SGD, 60 epochs), validation accuracy (80%), per-class confusion matrix, per-domain accuracy (77–81%), robustness under Gaussian feature perturbation (Figure 11), and a predictor-quality ablation (Table 6).

The predictor (`edge_simulator.ComplexityPredictor`) and its trained weights are released in the repository linked from Section 5.

Major 4 — Related-work expansion on MAB-based network routing

The related work section must be expanded to better situate this contribution within the broader literature on MAB-based network routing. In particular, the authors should discuss and cite recent work such as Scarvaglieri et al. ("MAGELLAN: A Distributed MAB-based Algorithm for Energy-Fair and Reliable Routing in Multi-Hop LoRa Networks," IEEE GLOBECOM 2024) and Lian et al. ("Intelligent Channel Allocation for IEEE 802.11be Multi-Link Operation: When MAB Meets LLMs," IEEE Journal on Selected Areas in Communications, Vol. 43, No. 11), which demonstrate the applicability of MAB frameworks to routing problems in different network environments and are directly relevant to the positioning of this work.

Section 2 has been expanded with a new paragraph on MAB-based network routing, and Table 1 has been extended to position these works against ours. Both suggested works are cited and discussed. The new paragraph reads (with modifications shown in the revised manuscript):

> A closely related body of work uses MAB formulations to optimise routing and resource allocation in wireless and multi-hop networks, where the reward-feedback structure—selecting one of several channels or paths whose quality is only observed after use—mirrors the edge-cloud LLM setting. Zhao et al. (2011) introduced combinatorial bandits with dependent arms for opportunistic network routing, and Tekin and van der Schaar (2015) extended contextual bandits to cooperative distributed settings. More recently, Scarvaglieri et al. (2024) propose MAGELLAN, a distributed MAB algorithm for energy-fair, reliable routing in multi-hop LoRa networks, and Lian et al. (2025) combine MAB with LLM-assisted channel selection for IEEE 802.11be multi-link operation, demonstrating that bandit-style adaptation scales to realistic wireless deployments. The present work differs from these approaches in the reward structure (quality scores rather than link throughput), the exploration unit (execution tier rather than wireless channel), and the explicit deadline-satisfaction term in the arm score; however, all share the motivation that online learning outperforms static policies under unpredictable conditions.

Both references have been added to the bibliography as `scarvaglieri2024magellan` and `lian2025multilink`. Table 1 (comparison with related work) has been extended to include these works and now has an additional "Tail-aware" column that distinguishes the present contribution.

Major 5 — Open-source simulator repository link

Finally, the paper mentions releasing an open-source simulator but provides no repository link. To enable reproducibility, the authors must publish the simulator source code along with the configuration files used in the experiments and the MLP classifier, and include a direct link to the repository. I would recommend this be added as a footnote at the point where the simulator is first introduced in Section 5.

The repository is available at https://github.com/Sangdon-Park/edgellm-router. It contains the full simulator (`edge_simulator.py`), the experiment scripts (`experiments_mab_routing.py`, `experiments_quality_validation.py`), the complexity-predictor code and trained weights, the configuration files used to produce every table and figure in the paper, and a `run_experiments.sh` entry point that reproduces the main comparison in a single invocation.

As recommended, a footnote link has been added at the first mention of the simulator in Section 5, and again in the introduction (contribution 3). The abstract also ends with the repository URL.

Closing

We thank the Editor and both Reviewers for feedback that substantially improved the manuscript. All changes are marked in the revised submission. We hope the revision addresses the concerns raised and are happy to make further adjustments where useful.